

COMPSCI 701:
Creating Maintainable Software
Design Quality 2

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

Agenda

- Agenda

- Preparation
- Finish 10.1
- Design Advice
- Smells
- SOLID
- Design Patterns
- Abstraction
- Key Points

- PAQ 10.2
- Admin
 - ???
- Lecture 10.3 preparation
- Finish 10.1
- How does different design advice related?

Class Discussion (Wednesday)

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- Smells
- SOLID
- Design Patterns
- Abstraction
- Key Points

- A good class is:
 - a good abstraction
 - highly cohesive
 - follows Single Responsibility Principle

Are these all the same? If not, how are they different?

- This week's class discussion addresses this question
- Read the definitions on Canvas before class
- Answer questions (on Qualtrics) about code implementing the “house” concept in Kalah

Finish 10.1

- Agenda
- Preparation
- **Finish 10.1**
- Design Advice
- Smells
- SOLID
- Design Patterns
- Abstraction
- Key Points

- Finish yesterday's lecture

- Agenda
- Preparation
- Finish 10.1
- **Design Advice**
- Smells
- SOLID
- Design Patterns
- Abstraction
- Key Points

Design Advice

- There's lots of design advice, but is it all telling us the same thing? Is it consistent?
- Good advice gives us some idea of how to improve design (actionable)
- Good advice should also be consistent with other good advice

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- **Smells**
- SOLID
- Design Patterns
- Abstraction
- Key Points

Smell: Duplicate Code

- Duplicate code can (“clones”) be a cause of faults, and its presence often indicates a potential design problem (or at least an opportunity for improving the design)
- If two code fragments are the “same”, and one has to change, then the other probably has to change as well. Not doing so may introduce a fault
- If you need some code fragment twice, chances are you need it more than twice, so changing your design to remove the duplication will likely support “extensibility”
- Also Known As DRY — “Don’t Repeat Yourself”
- if one of the clones has to change then they probably all should, increasing the number of change cases where all classes containing the clones need to change
⇒ increasing $\mathcal{CR}$

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- **Smells**
- SOLID
- Design Patterns
- Abstraction
- Key Points

Example

```
class KiwiMoney {
    private int _dollars;
    private int _cents;
    public KiwiMoney(int dollars , int cents) {
        _dollars = dollars;
        _cents = cents;
    }
    public String toString() {
        return "$" + _dollars + "." + Utility.pad(_cents,2);
    }
    public String formalString() {
        return _dollars + "." + Utility.pad(_cents,2) + "NZD";
    }
}
```

```
class AussieMoney {
    private int _dollars;
    private int _cents;
    public AussieMoney(int dollars, int cents) {
        _dollars = dollars;
        _cents = cents;
    }
    public String toString() {
        return "$" + _dollars + "." + Utility.pad(_cents,2);
    }
    public String formalString() {
        return _dollars + "." + Utility.pad(_cents,2) + "AUD";
    }
}
```

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- **Smells**
- SOLID
- Design Patterns
- Abstraction
- Key Points

Example

```
abstract class AbstractMoney {
    private int _dollars;
    private int _cents;
    public AbstractMoney(int dollars , int cents) {
        _dollars = dollars;
        _cents = cents;
    }
    public String toString() {
        return "$" + _dollars + "." + Utility.pad(_cents,2);
    }
    public String formalString() { // template method
        return _dollars + "." + Utility.pad(_cents,2) + getCurrencyCode();
    }
    abstract public String getCurrencyCode();
}
```

```
class KiwiMoney extends AbstractMoney {
    public KiwiMoney(int dollars, int cents) { super(dollars, cents); }
    public String getCurrencyCode() { return "NZD"; }
}
class AussieMoney extends AbstractMoney {
    public AussieMoney(int dollars, int cents) { super(dollars, cents); }
    public String getCurrencyCode() { return "AUD"; }
}
```


- Agenda
- Preparation
- Finish 10.1
- Design Advice
- Smells
- SOLID
- Design Patterns
- Abstraction
- Key Points

Reduce Duplication, Improve Design

- Introduce AbstractMoney as parent to both classes to reduce duplication
- If using inheritance, then must be able to substitute KiwiMoney or AussieMoney for AbstractMoney $\Rightarrow$ LSP
- Once we have AbstractMoney, we can create USMoney without changing AbstractMoney $\Rightarrow$ OCP
- Total number of classes (N) has gone up, number of classes that need to change (for same change cases) has gone down (C) $\Rightarrow \mathcal{CR}$ has decreased $\Rightarrow$ alterability has improved

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- **Smells**
- SOLID
- Design Patterns
- Abstraction
- Key Points

Favour Composition over Inheritance

```
class Money {
    private int _dollars;
    private int _cents;
    private String _currencyCode;
    public KiwiMoney(int dollars , int cents, String currencyCode) {
        _dollars = dollars;
        _cents = cents;
        _currencyCode = currencyCode;
    }
    public String toString() {
        return "$" + _dollars + "." + Utility.pad(_cents,2);
    }
    public String formalString() {
        return _dollars + "." + Utility.pad(_cents,2) + _currencyCode;
    }
}
```

- Inheritance can be used to reduce duplication
- But it is not the only form of “reuse”
- “Favor Composition over Inheritance” is advice given by Gamma (of the Design Patterns Gang of Four).

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- **Smells**
- SOLID
- Design Patterns
- Abstraction
- Key Points

Duplicate Code?

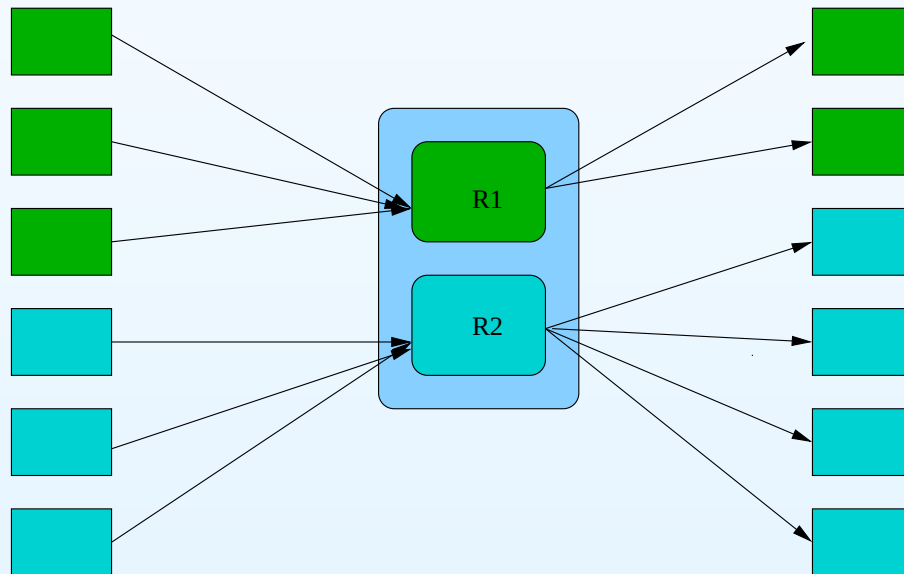
```
public class Supplier {  
    private String _id;  
    public String getID() {  
        return _id;  
    }  
    // other Supplier stuff  
}  
  
public class Part {  
    private String _id;  
    public String getID() {  
        return _id;  
    }  
    // other Part stuff  
}
```

- There's “pseudo” duplication — the nature of what needs to be done means the solution looks the same
 - changing in one place does not require changing in the other
- Suppose we created a common parent, e.g ID. Would we ever want to substitute `Supplier` or `Part` for `ID`?
 - No — don't create the common parent (LSP)
 - Yes — create the common parent (LSP)
- Good design advice gives you a way to evaluate different design choices — actionable.

SRP

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- Smells
- **SOLID**
- Design Patterns
- Abstraction
- Key Points

- Classes that have more than one responsibility likely have more classes that they depend on (for each responsibility) than those that follow SRP (high FAN-OUT)
- Classes that have more than one responsibility likely have more classes that depend on them (for each responsibility) than those that follow SRP (high FAN-IN)



- **Non-actionable advice:** Reduce FAN-OUT
- **Actionable advice:** Moving different responsibilities into different classes

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- Smells
- **SOLID**
- Design Patterns
- Abstraction
- Key Points

SRP versus Cohesion

- Cohesion: “the extent to which its individual components are needed to perform the **same task**” — SRP?
- Cohesion: “how tightly bound or related the internal elements are to one another” — internal elements could be tightly bound but provide more than one responsibility?
- SRP: “Gather together the things that change for the same reasons. Separate those things that change for different reasons” — if tightly bound then changing one element means changing other elements?

LSP

- The **context** code expects objects from a particular class A
- Objects from B can be *substituted* if B is a descendant of A *and* LSP is followed (i.e. substituting B for A will not break anything)
- Therefore the coupling between **context** and the other classes is not particularly strong — strong coupling on something implies if the something is not available something doesn't work

```
class Reporter {
    public void report(AbstractMoney money) {
        // Context – behaves differently but never changes, so is reused
        // Depends on AbstractMoney, but not KiwiMoney, AussieMoney, USMoney
        System.out.println("I got " + money.formalString());
    }
}

class Main {
    public static void main(String[] args) {
        Reporter currentAccounts = new Reporter();
        AbstractMoney myMoney = new KiwiMoney(100,0);
        currentAccounts.report(myMoney); // substitution
        myMoney = new AussieMoney(50,0);
        currentAccounts.report(myMoney); // more substitution
        myMoney = new USMoney(10,0);
        currentAccounts.report(myMoney); // still substituting
    }
}
```

ISP

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- Smells
- **SOLID**
- Design Patterns
- Abstraction
- Key Points

- The more one class knows about another (such as its interface) the stronger the coupling

DIP

- Depending on a concrete thing has higher coupling than depending on an abstract thing

```
interface MovieFinder {
    List<Movie> findAll();
}
class MovieLister {
    // MovieLister depends on abstract MovieFinder
    // not concrete CVSMovieFinder
    private MovieFinder _finder = null;

    public MovieLister(MovieFinder finder) {
        _finder = finder;
    }

    public List<Movie> moviesDirectedBy(String dir) {
        List<Movie> allMovies = _finder.findAll();
        // iterate through 'allMovies' and find those with
        // 'dir' as director...
    }
}
```

number of types coupled to has not decreased, but strength of coupling has decreased

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- Smells
- SOLID
- Design Patterns
- Abstraction
- Key Points

Template Method

- The template method has low coupling to the specifics of the “hook” methods it uses

```
public abstract class TestCase extends Assert implements Test {
    public void runBare() throws Throwable {
        Throwable exception = null;
        setUp();
        try {
            runTest();
        } catch (Throwable running) {
            exception = running;
        } finally {
            try {
                tearDown();
            } catch (Throwable tearingDown) {
                if (exception == null) exception = tearingDown;
            }
        }
        if (exception != null) throw exception;
    }
    ...
    protected void setUp() throws Exception

    protected void tearDown() throws Exception
}
```

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- Smells
- SOLID
- Design Patterns
- Abstraction
- Key Points

Observer

- The observer has low coupling on what it observes
- The observed has low coupling on its observer

Factory Method

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- Smells
- SOLID
- Design Patterns
- Abstraction
- Key Points

- The consumer of the product has low coupling on the product it consumes

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- Smells
- SOLID
- Design Patterns
- **Abstraction**
- Key Points

Abstraction

- Much of the advice seems to suggest “Use abstraction as much as possible”
 - “Good” classes are those that behave like the concepts they are meant to represent:

“This is especially true if we take an object-oriented view of the world, because objects, as **abstractions of entities in the real world**, represent a particularly dense and cohesive clustering of information.” Booch
- But consider the code smell **Speculative Generality**
 - designs that use abstractions that aren’t used
 - abstractions are harder to understanding (and generally, maintain)
 - so there should be a clear **benefit** to offset the **cost**
- **Much more research needed into how to reliably determine benefit and cost**

- Agenda
- Preparation
- Finish 10.1
- Design Advice
- Smells
- SOLID
- Design Patterns
- Abstraction
- Key Points

Key Points

- Measurement of software is really really complicated, so be suspicious of any proposed software metrics, especially any that claim to measure quality, because (a) they don't and (b) they're not useful for making design decisions
- Coupling and Cohesion are frequently mentioned as useful attributes of design quality
- If any of the design advice has any value, there must be relationships between it all. e.g. design smells, design patterns, and SOLID principles
- Abstraction is good, but like anything, there can be too much of a good thing